

Ereditarietà in Python

Ereditarietà in Python

Come creare classi basate su altre classi in Python.

Perché è utile usare l'ereditarietà

Immagina di dover descrivere gli animali in un programma: cani, gatti, uccelli. Tutti respirano, tutti mangiano — ma ognuno fa un suono diverso. Dovresti riscrivere il codice per "respira" e "mangia" tre volte?

L'**ereditarietà** risolve questo problema: scrivi il codice comune una volta sola in una classe "genitore", e le classi "figlie" lo ereditano automaticamente — aggiungendo solo le loro particolarità.

La metafora della famiglia

Pensa all'ereditarietà biologica:

- Un figlio eredita dai genitori molte caratteristiche (colore degli occhi, gruppo sanguigno)
- Ma ha anche caratteristiche sue proprie
- Può anche avere caratteristiche simili ai genitori ma con variazioni (stesso naso, ma di misura diversa)

In programmazione funziona esattamente così.

Sintassi di base

```
class ClassePadre:
    # metodi e attributi del genitore

class ClasseFiglia(ClassePadre):    # ← il nome tra parentesi indica
    "eredita da"
    # eredita tutto dal padre
    # può aggiungere roba nuova o modificare ciò che eredita
```

Esempio: animali

```
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def respira(self):
        print(f"{self.nome} respira.")

    def mangia(self):
        print(f"{self.nome} mangia.")

# Cane eredita da Animale: ha tutti i suoi metodi + "abbaia"
class Cane(Animale):
    def abbaia(self):
        print(f"{self.nome} fa: Bau bau!")

# Gatto eredita da Animale: ha tutti i suoi metodi + "fa_le_fusa"
class Gatto(Animale):
    def fa_le_fusa(self):
        print(f"{self.nome} fa le fusa...")

fido = Cane("Fido")
fido.respira()      # → Fido respira.    (metodo ereditato da Animale)
fido.mangia()       # → Fido mangia.     (metodo ereditato da Animale)
fido.abbaia()       # → Fido fa: Bau bau! (metodo proprio di Cane)

micio = Gatto("Micio")
micio.respira()     # → Micio respira.
micio.fa_le_fusa()  # → Micio fa le fusa...
```

Sovrascrivere i metodi (Override)

La classe figlia può **ridefinire** un metodo del genitore per cambiarne il comportamento. È come un figlio che ha "lo stesso tipo di carattere" del padre, ma con sfumature diverse.

```
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def suono(self):
        print("...") # Suono generico

class Cane(Animale):
    def suono(self): # Sovrascrive il metodo del genitore
        print(f"{self.nome} fa: Bau bau!")

class Gatto(Animale):
    def suono(self): # Sovrascrive il metodo del genitore
        print(f"{self.nome} fa: Miao!")

fido = Cane("Fido")
micio = Gatto("Micio")

fido.suono() # → Fido fa: Bau bau!
micio.suono() # → Micio fa: Miao!
```

super() : richiamare il genitore

A volte non vuoi **sostituire** completamente un metodo del genitore, ma vuoi **estenderlo** — aggiungere comportamento in più. `super()` ti permette di chiamare il metodo originale del genitore.

```
class Animale:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def presenta(self):
        print(f"Mi chiamo {self.nome} e ho {self.eta} anni.")

class Cane(Animale):
    def __init__(self, nome, eta, razza):
        super().__init__(nome, eta)  # Chiama __init__ del genitore per i
        # dati comuni
        self.razza = razza           # Aggiunge il dato extra "razza"

    def presenta(self):
        super().presenta()           # Prima esegui la presentazione del
        # genitore...
        print(f"Sono un {self.razza}.")  # ...poi aggiungi info in più

fido = Cane("Fido", 3, "Labrador")
fido.presents()
# → Mi chiamo Fido e ho 3 anni.
# → Sono un Labrador.
```

Verificare la parentela tra classi e oggetti

```
class Animale:
    pass

class Cane(Animale):
    pass

fido = Cane()

# isinstance() verifica se un oggetto è di un certo tipo (o di un suo
# "antenato")
print(isinstance(fido, Cane))    # True ← fido è un Cane
print(isinstance(fido, Animale)) # True ← fido è anche un Animale (per
# ereditarietà)

# issubclass() verifica se una classe è "figlia" di un'altra
print(issubclass(Cane, Animale)) # True ← Cane è una sottoclasse di
Animale
```

Esempio pratico: forme geometriche

```
class Forma:
    def __init__(self, colore):
        self.colore = colore

    def descrivi(self):
        print(f"Sono una forma {self.colore}.")

# Rettangolo eredita da Forma
class Rettangolo(Forma):
    def __init__(self, base, altezza, colore):
        super().__init__(colore)      # Passa il colore al genitore
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza

# Cerchio eredita da Forma
class Cerchio(Forma):
    def __init__(self, raggio, colore):
        super().__init__(colore)      # Passa il colore al genitore
        self.raggio = raggio

    def area(self):
        return 3.14159 * self.raggio ** 2

r = Rettangolo(5, 3, "rosso")
r.descrivi()      # → Sono una forma rosso.    (metodo ereditato da Forma)
print(r.area())   # → 15

c = Cerchio(4, "blu")
c.descrivi()      # → Sono una forma blu.
print(c.area())   # → 50.265...
```

Entrambe le forme condividono il metodo `descrivi()` — scritto una volta sola nel genitore — e ognuna ha la sua formula per calcolare l'area.